



**Universidad
Nacional
de San Martín**

UNIVERSIDAD NACIONAL DE SAN MARTÍN

EXAMEN FINAL DE PYTHON

FUNCIONES, POO, DECORADORES, REQUESTS Y ANÁLISIS DE DATOS

Instrucciones

- Duración: 2 horas.
- Se evaluará la correcta implementación de funciones, uso de estructuras de datos, comprensión de POO, decoradores, requests, manejo de archivos, excepciones y análisis de datos con `numpy`, `matplotlib` y `pandas`.
- Se descontarán puntos por errores de sintaxis, falta de claridad o no cumplir con el formato pedido de input y output.
- Justificá brevemente tus respuestas teóricas cuando se solicite.

Ejercicio 1 – Gestión de pedidos (25 pts)

Se tiene un sistema simple para registrar pedidos de una pizzería.

1.a) Preguntas teóricas (7 pts)

- (i) Explica con tus palabras la diferencia entre **lista** y **tupla** en Python. Incluí un ejemplo de cada una. (4 pts)
- (ii) ¿Qué es un **diccionario** en Python y en qué situación lo usarías en lugar de una lista? (3 pts)

1.b) Procesamiento de pedidos (18 pts)

Dado el siguiente listado de pedidos:

```
1 pedidos = [  
2     {"id": 1, "producto": "Muzzarella", "cantidad": 2, "precio_unitario": 12000, "modo": "Delivery"  
3     },  
4     {"id": 2, "producto": "Fugazzeta", "cantidad": 1, "precio_unitario": 14000, "modo": "Local"},  
5     {"id": 3, "producto": "Calabresa", "cantidad": 3, "precio_unitario": 15000, "modo": "Delivery"},  
6     {"id": 4, "producto": "Muzzarella", "cantidad": 1, "precio_unitario": 12000, "modo": "Local"},  
7 ]
```

- (i) Implementa una función `total_pedido(pedido)` que reciba un diccionario con la estructura anterior y devuelva el total de ese pedido (`cantidad * precio_unitario`). (4 pts)
- (ii) Usando **comprensiones de listas**, construí una lista llamada `totales` con el total de cada pedido. (4 pts)
- (iii) Implementa una función `total_por_producto(pedidos)` que devuelva un diccionario donde las claves sean los nombres de producto y los valores el total facturado por cada uno. Ejemplo de salida:

```
{"Muzzarella": 48000, "Fugazzeta": 14000, "Calabresa": 45000}
```

(6 pts)

- (iv) Escribí una expresión con `sorted` y una **lambda** para obtener una lista de productos ordenados de mayor a menor según su total facturado. (4 pts)

Ejercicio 2 – Sistema de vehículos de reparto (25 pts)

2.a) Preguntas teóricas (8 pts)

- (i) Nombra y explica brevemente **dos pilares** de la Programación Orientada a Objetos (por ejemplo: encapsulamiento, herencia, polimorfismo, abstracción). Inclú un ejemplo muy simple de código (en pseudocódigo o Python) que ilustre uno de ellos. (4 pts)
- (ii) ¿Qué es un **decorador** en Python y para qué podría utilizarse en un sistema real? Menciona al menos un caso de uso. (4 pts)

2.b) Clases y herencia (10 pts)

Se quiere modelar la flota de vehículos de reparto de la pizzería.

- (i) Implementa una clase **Vehiculo** con los atributos: **patente**, **tipo** (por ejemplo “moto”, “auto”, “camioneta”) y **anio**. Agregá un método **descripcion()** que devuelva un **string** con el formato: `"Vehiculo ABC123 - moto (2018)"`. (5 pts)
- (ii) Crea una subclase **MotoReparto** que herede de **Vehiculo** y agregue el atributo **caja_termica** (booleano). Sobrescribí el método **descripcion()** para que indique si tiene caja térmica o no. (5 pts)

2.c) Decorador de validación (7 pts)

Implementa un decorador `@verificar_anio_minimo` que, aplicado a una función de registro de vehículos, impida registrar vehículos con más de 12 años de antigüedad. En ese caso, debe lanzar una excepción personalizada **VehiculoAntiguoError** con un mensaje claro. (7 pts)

Ayuda: Podés suponer que la función decorada recibe como argumento un objeto **Vehiculo** o los datos necesarios para calcular la antigüedad.

Ejercicio 3 – Monitoreo de una API pública (25 pts)

Queremos consultar periódicamente una API pública que devuelve información de una pizzería ficticia en formato JSON.

3.a) Preguntas teóricas (8 pts)

- (i) Explica brevemente para qué sirve el bloque `try / except` en Python. ¿Cuál es la diferencia entre capturar una excepción específica (por ejemplo `ValueError`) y capturar cualquier excepción genérica con `except Exception`? (4 pts)
- (ii) ¿Qué ventajas tiene guardar las respuestas de una API en un archivo (por ejemplo `.json`) en lugar de mantener toda la información solo en memoria? Menciona al menos dos ventajas. (4 pts)

3.b) Consulta a la API y guardado en archivo (10 pts)

- (i) Escribe una función `consultar_api_pizzeria(url, n)` que realice `n` solicitudes HTTP GET a la URL dada usando la librería `requests`, esperando 2 segundos entre cada una con `time.sleep(2)`. La función debe:
 - Devolver una lista con todas las respuestas (parseadas a Python con `response.json()`).
 - Manejar posibles errores de conexión con `try / except`, mostrando un mensaje claro si la consulta falla.(6 pts)
- (ii) Implementa una función `guardar_respuestas(rutas, nombre_archivo)` que guarde la lista de respuestas en un archivo `.json`. Usá manejo de excepciones para:
 - Capturar errores al abrir o escribir el archivo.
 - Mostrar un mensaje de error sin que el programa se interrumpa abruptamente.(4 pts)

3.c) Carga desde archivo y filtrado con generadores (7 pts)

- (i) Escribe una función `cargar_respuestas(nombre_archivo)` que lea el archivo `.json` generado y devuelva la lista de respuestas. Debe manejar el caso de `FileNotFoundError` con un mensaje claro. (3 pts)
- (ii) Implementa un **generador** `filtrar_por_total(respuestas, umbral)` que reciba la lista de respuestas y vaya devolviendo solo aquellas cuya venta total del pedido (suponiendo que en el JSON viene un campo `"total"`) sea mayor al umbral. (4 pts)

Ejercicio 4 – Análisis de ventas de la pizzería (25 pts)

Se cuenta con un archivo `ventas.pizzeria.final.xlsx` con el registro de ventas de un mes. Columnas: `Fecha`, `Producto`, `Categoría`, `Cantidad`, `Precio_Unitario`, `Modo_Venta`, `Sucursal`.

4.a) Preguntas teóricas (7 pts)

- (i) ¿Cuál es la diferencia entre una `Series` y un `DataFrame` en `pandas`? (3 pts)
- (ii) ¿Para qué se utiliza normalmente `numpy` al trabajar junto con `pandas` y `matplotlib`? Menciona al menos dos usos. (4 pts)

4.b) Numpy y Matplotlib (8 pts)

- (i) Usando `numpy`, generá 100 valores entre 0 y 4π y calculá `np.sin(x)` y `np.cos(x)`. (3 pts)
- (ii) Graficá ambas funciones en una misma figura usando `matplotlib`, con título, nombre de ejes y leyenda que identifique cada curva. (5 pts)

4.c) Análisis con Pandas (10 pts)

- (i) Carga el archivo en un `DataFrame` llamado `df` y muestra las primeras 5 filas. (2 pts)
- (ii) Crea una nueva columna `Total` como `Cantidad * Precio_Unitario`. (2 pts)
- (iii) Calcula el total vendido por `Categoría` y guardalo en un nuevo `DataFrame` llamado `df_categorias`. (3 pts)
- (iv) Genera un gráfico de barras que muestre el total vendido por categoría utilizando `df_categorias`. Incluí título y etiquetas en los ejes. (3 pts)